

کتاب مقدس

- سطح: آسان
- طراح: حدیث صابری

در یک کتابخانه جادویی، یک **کتاب مقدس** وجود دارد که فقط **کتابداران مجاز** می‌توانند آن را تغییر دهند یا حذف کنند، اما **همه مردم** می‌توانند آن را بخوانند. شما باید سیستمی طراحی کنید که:

۱. فقط یک نسخه از کتابخانه در کل برنامه وجود داشته باشد.
۲. دسترسی به کتاب مقدس توسط یک **نماینده** کنترل شود.

می‌توانید کلاس‌های اولیه پروژه را از [این لینک](#) دانلود کنید.

جزئیات پروژه

کلاس SacredBook

این کلاس نشان‌دهنده **کتاب مقدس** است.

ویژگی‌ها (fields):

- content (از نوع String): محتوای کتاب

متدها:

- SacredBook(String content) : سازنده - مقدار اولیه محتوا را می‌گیرد.
- String getContent() : محتوای کتاب را برمی‌گرداند.
- void setContent(String newContent) : محتوای کتاب را تغییر می‌دهد.
- void clear() : محتوای کتاب را خالی می‌کند (به " " تبدیل می‌کند).

کلاس Librarian

این کلاس نشان‌دهنده **کتابداری** است که می‌خواهد به کتاب دسترسی پیدا کند.

ویژگی‌ها (fields):

- name (از نوع String): اسم کتابدار
- isAuthorized (از نوع boolean): آیا این کتابدار مجاز به تغییر کتاب است؟

متدها:

- Librarian(String name, boolean isAuthorized): سازنده
- String getName(): اسم را برمی‌گرداند
- boolean isAuthorized(): وضعیت مجوز را برمی‌گرداند.

اینترفیس BookAccess

این یک قرارداد است که مشخص می‌کند چه کارهایی می‌توان روی کتاب انجام داد.

متدها:

- String read()
- void write(String newContent)
- void delete()

کلاس RealBookAccess

این کلاس **کار واقعی** را روی کتاب انجام می‌دهد (بدون هیچ کنترلی).

ویژگی‌ها:

- یک reference به SacredBook

متدها: (پیاده‌سازی اینترفیس BookAccess)

- read(): مقدار getContent() را برمی‌گرداند.
- write(String newContent): مقدار setContent(newContent) را صدا می‌زند.
- delete(): مقدار clear() را صدا می‌زند.

کلاس BookAccessProxy

این کلاس **نماینده** است و کنترل دسترسی را انجام می‌دهد.

ویژگی‌ها:

- librarian : کتابداری که درخواست می‌دهد.
- realAccess : شیء واقعی که کارها را انجام می‌دهد.

متدها: (پیاده‌سازی اینترفیس BookAccess)

- دقت کنید که همه می‌توانند بخوانند، ولی فقط کتابدار مجاز می‌تواند بنویسد یا پاک کند.
- اگر کتابدار مجاز نبود و خواست بنویسد یا پاک کند، پیام "Access denied for [name]" چاپ شود و عملیات انجام نشود.

کلاس LibraryManager

این کلاس مسئول **مدیریت کتابخانه** است و فقط **یک نمونه** از آن وجود دارد.

ویژگی‌ها:

- book : یک شیء از نوع SacredBook .
- instance : یک استاتیک برای نگهداری نمونه.

متدها:

- LibraryManager() : سازنده.
- static LibraryManager getInstance() : نمونه را برمی‌گرداند (اگر نبود، می‌سازد).
- SacredBook getBook() : کتاب را برمی‌گرداند.
- void setBook(SacredBook book) : کتاب را تنظیم می‌کند.

مثال

ورودی نمونه ۱

```
public class Main {  
    public static void main(String[] args) {  
  
        LibraryManager manager = LibraryManager.getInstance();  
  
        SacredBook book = new SacredBook("Ancient Wisdom");  
  
        manager.setBook(book);  
  
        RealBookAccess realAccess = new RealBookAccess(manager.getBook());  
  
        Librarian ali = new Librarian("Ali", false);  
  
        Librarian reza = new Librarian("Reza", true);  
  
        BookAccessProxy proxyAli = new BookAccessProxy(ali, realAccess);  
  
        BookAccessProxy proxyReza = new BookAccessProxy(reza, realAccess);  
  
        System.out.println(proxyAli.read());  
  
        proxyAli.write("Hacked");  
  
        System.out.println(proxyAli.read());  
  
        proxyReza.write("Real Secret");  
  
        System.out.println(proxyReza.read());  
  
        proxyReza.delete();  
  
        System.out.println(proxyReza.read());  
    }  
}
```

خروجی نمونه ۱

Ancient Wisdom
Access denied for Ali
Ancient Wisdom
Real Secret

آنچه که باید آپلود کنید

شما باید یک فایل zip آپلود کنید که شامل کلاس های زیر باشد:

SacredBook.java
Librarian.java
BookAccess.java
RealBookAccess.java
BookAccessProxy.java
LibraryManager.java

easyPay

- سطح: متوسط
- طراح: دانیال یاقوتی

شرکت easyPay از شما درخواست کرده تا در طراحی سیستم مدیریت بلیط هوشمند مترو کمکش کنید. وظیفه شما پیاده سازی این پروژه با استفاده از "الگو های طراحی" (Design Patterns) مناسب است.

ساختار فایل ها

```
fileName.zip
├─ MetroCard.java
├─ MetroSystem.java
├─ MetroAccount.java
├─ ProcessesTyep.java
├─ App.java
└─ Observer.java
```

برای دانلود فایل اولیه پروژه کلیک کنید 📦

جزئیات پروژه

کلاس MetroCard :

این کلاس نمایانگر یک کارت فیزیکی مترو با شناسه یکتا است.

فیلد	نوع	توضیحات
id	String	شناسه یکتا هر کارت

سازنده:

```
1 | public MetroCard(String id)
```

فیلد مربوطه را مقدار دهی می‌کند.

کلاس Observer (از نوع اینترفیس):

یکی از الگوهای طراحی مرسوم Observer نام دارد که اجازه اطلاع رسانی تغییرات به اشیا مختلف را فراهم می‌کند.

متدها:

1. متد update :

```
1 | public abstract void update(String message);
```

در کلاس مقصد مشخص می‌شود که با پیام دریافتی باید چه عملیاتی انجام شود.

کلاس MetroAccount :

برای جلوگیری از تغییرات مستقیم و ذخیره نکردن موجودی به طور مستقیم روی کارت مترو، متناظر با هر کارت فیزیکی یک حساب کاربری ساخته می‌شود. مدیریت و اعمال تغییرات روی این حساب توسط کلاس MetroSystem صورت می‌گیرد که در ادامه معرفی می‌شود. (الگوی Proxy).

فیلد	نوع	توضیحات
id	String	شناسه یکتا هر حساب
balance	double	موجودی
[]observers	Observers	دریافت کننده های تغییرات

سازنده:

```
1 | public MetroAccount(String id)
```

- فیلد id را مقدار دهی می‌کند.
- مقدار balance به طور پیش فرض \$2.0 مقدار دهی می‌شود.

- طول ابتدایی آرایه `observers` را 5 واحد مقدار دهی کنید و در صورت نیاز به صورت پویا افزایش دهید.

##متدها: 1. متد `addObserver` :

```
1 | public void addObserver(Observer observer)
```

- یک دریافت کننده به لیست دریافت کننده های حساب اضافه می کند.

2. متد `charge` :

```
1 | public boolean charge(double amount)
```

- در صورت معتبر بودن مقدار `amount` آن را به موجودی اضافه می کند و به تمام دریافت کننده های با استفاده از متد `notifyObservers` (در ادامه معرفی می شود) به دریافت کننده ها اطلاع داده می شود؛ مقدار `true` را برمی گرداند.
- در غیر این صورت مقدار `false` را برمی گرداند.

3. متد `deduct` :

```
1 | public boolean deduct(double amount)
```

- در صورت معتبر بودن مقدار `amount` و امکان کسر از موجودی، از شارژ حساب کاربری کاسته و به تمام دریافت کننده ها اطلاع می دهد؛ در نهایت مقدار `true` را برمی گرداند.
- در غیر این صورت مقدار `false` را برمی گرداند.

4. متد `showBalance` :

```
1 | public void showBalance()
```

- مقدار موجودی حساب را با استفاده از متد `notifyObservers` را به دریافت کننده ها اطلاع می دهد.

5. متد notifyObservers :

```
1 | public void notifyObservers(ProcessesType type, double amount)
```

- یک **ProcessesType** دریافت می‌کند از نوع enum بوده و مشخص کننده سه نوع فرایند زیر می‌باشد:

۱. شارژ حساب: CHARGE

۲. کسر از موجودی: DEDUCT

۳. نمایش موجودی: SHOW_BALANCE

- یک مقدار amount دریافت می‌کند که بسته به زمان فراخوانی می‌تواند مقدار افزوده شده به شارژ حساب، موجودی یا مقدار کسر شده باشد.
- وظیفه شما این است که یک پیغام مناسب برای اطلاع رسانی به دریافت کننده‌ها با توجه به نوع فرایند ورودی به شکل زیر بسازید:

```
Card [id] [status]: [amount]
```

- status وابسته به تایپ فرایند ورودی می‌تواند یکی از وضعیت های "Charged" و "Deducted" و "Balance" باشد.
- مقدار amount را تا دو رقم اعشار نمایش دهید.
- در نهایت به تمام دریافت کننده های موجود در لیست observers با متد update اطلاع داده شود.

کلاس MetroSystem :

برای اینکه تمام دسترسی های به این سیستم یکتا یکسان باشد، این کلاس را با استفاده از الگوری singleton پیاده سازی می‌کنیم.

##دریافت نمونه واحد:

```
1 | public static MetroSystem getInstance()
```

یک شی واحد از نوع MetroSystem برمی‌گرداند.

فیلد	نوع	توضیحات
accounts[]	MetroAccount	لیست حساب های کاربری ثبت شده

##سازنده:

```
1 | private MetroSystem()
```

سازنده private این کلاس که آرایه accounts را با مقدار اولیه 10 مقدار دهی می‌کند و در صورت نیاز به طور پویا طول آن را افزایش دهید.

##متدها:

1. متد nextID :

```
1 | private String nexID()
```

- هر بار که صدا زده می‌شود یک شناسه یکتا به فرمت MTC-number تولید می‌کند که number از 0 شروع شده و بعد از ساخت هر شناسه 1 واحد افزایش پیدا می‌کند. برای مثال، پنجمین باری که این تابع فراخوانی میشه باید شناسه MTC-4 را تولید کند.

2. متد registerMetroCard :

```
1 | public MetroCard registerMetroCard()
```

- یک MetroCard و متناظر با آن یک MetroAccount با شناسه های یکسان می‌سازد.
- برای ساخت شناسه از متد قبلی استفاده کنید.

3. متد addAccount :

```
1 | private void addAccount(MetroAccount account)
```

- یک Metro Account را به حساب های کاربری سیستم اضافه می کند.

4. متد authorize :

```
1 | public boolean authorize(MetroCard metroCard)
```

- یک کارت مترو ورودی می گیرد و بررسی می کند آیا این کارت معتبر است یا خیر؛ اگر بله مقدار true و اگر خیر مقدار false برمی گرداند.

5. متد useCard :

```
1 | public boolean useCard(MetroCard metroCard)
```

- یک کارت مترو ورودی می گیرد در صورت معتبر بودن کارت و امکان برداشت هزینه ی بلیط مترو مقدار true را و در غیر این صورت false را برمی گرداند.
- هزینه ی هر بلیط مترو مبلغ \$2.0 می باشد.

6. متد chargeCard :

```
1 | public boolean chargeCard(MetroCard metroCard, double amount)
```

- در صورت معبر بودن کارت مترو، سعی میکند کارت مترو را به اندازه amount شارژ کند. در صورت موفقیت مقدار true و در غیر این صورت مقدار false را برمی گرداند.

7. متد showBalance :

```
1 | public void showBalance(MetroCard metroCard)
```

- یک کارت مترو را دریافت می کند و در صورت معتبر بودن متد showBalance را روی حساب کاربری متناظر با آن صدا می کند.

8. متد addAppObserver :

```
1 | public boolean addAppObserver(App app)
```

- یک شی App (در ادامه معرفی می‌شود) را دریافت می‌کند و آن را به عنوان یک Observer به حساب کاربری متناظر با Metro Card درون app اضافه می‌کند.

کلاس App :

این کلاس که رابط Observer را پیاده سازی می‌کند، نمایانگر اپلیکیشن اطلاع رسانی به هر کاربر است.

فیلد	نوع	توضیحات
name	String	نام دستگاه
MetroCard	metroCard	کارت مترو ثبت شده در اپلیکیشن
String	cache	آخرین پیغام دریافت شده

##سازنده:

```
1 | public App(String name)
```

- فیلد مربوطه را مقدار دهی می‌کند.

##متدها:

1. متد setMetroCard :

```
1 | public void setMetroCard(MetroCard metroCard)
```

- کارت مترو دریافتی را به عنوان metroCard فعال قرار می‌دهد.
- خود app را به عنوان observer به حساب کاربری متناظر با مترو کارت ثبت شده اضافه می‌کند.

```
1 | public void update(String message)
```

- مقدار cache را برابر message دریافتی قرار می‌دهد.

- پیغام زیر را چاپ می‌کند:

App Notification [name]: [message]

##مثال:

##ورودی:

```
1 public class Main {
2
3     public static void main(String[] args) {
4
5         MetroSystem metro = MetroSystem.getInstance();
6
7         MetroCard cardA = metro.registerMetroCard();
8
9         MetroCard cardB = metro.registerMetroCard();
10
11        System.out.println(cardA.getId());
12
13        System.out.println(cardB.getId());
14
15        System.out.println(metro.useCard(cardA));
16
17        System.out.println(metro.useCard(cardA));
18
19        System.out.println(metro.authorize(new MetroCard("THT-0")));
20
21        App app = new App("Mobile");
22
23        app.setMetroCard(cardA);
24
25        metro.chargeCard(cardA, 3.0);
26
27        metro.useCard(cardA);
28
29        System.out.println(metro.authorize(new MetroCard("MTC-7")));
30
31        App app2 = new App("Web");
32
33        app2.setMetroCard(cardA);
```

```
        metro.chargeCard(cardA, 3.0);

        metro.showBalance(cardA);

    }

}
```

###خروجی:

```
MTC-0
MTC-1
true
false
false
App Notification Mobile: Card MTC-0 Charged: 3.00
App Notification Mobile: Card MTC-0 Deducted: 2.00
false
App Notification Mobile: Card MTC-0 Charged: 3.00
App Notification Web: Card MTC-0 Charged: 3.00
App Notification Mobile: Card MTC-0 Balance: 4.00
App Notification Web: Card MTC-0 Balance: 4.00
```

چرخش و پوشش ماتریس نامنظم

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت
- سطح: متوسط
- طراح: امیرحسین علی پور

یک ماتریس نامنظم (ragged) به شما داده می‌شود که به صورت رشته‌هایی از ارقام نمایش داده می‌شود. هر رشته یک سطر را نشان می‌دهد و طول هر رشته می‌تواند متفاوت باشد. شما باید ورودی را بخوانید، آرایه نامنظم اعداد صحیح را بسازید و سه عملیات چرخش همراه با پوشش (mask) روی آن انجام دهید.

قالب ورودی:

۱. خط اول شامل یک عدد صحیح N است — تعداد سطرها.

۲. N خط بعدی هر کدام شامل یک رشته از ارقام (۰ تا ۹) می‌باشد. هر رشته نمایانگر یک سطر از ماتریس است و هر کاراکتر یک رقم است که باید به عدد صحیح تبدیل شود.

مثال ورودی:

```
3
123
45
6789
```

این ورودی معادل آرایه نامنظم زیر است:

```
[
  [1, 2, 3],
  [4, 5],
  [6, 7, 8, 9]
]
```

عملیات‌های چرخش: شما باید سه متد چرخش را پیاده‌سازی کنید:

- rotate90 : چرخش ماتریس به اندازه ۹۰ درجه در جهت عقربه‌های ساعت.
- rotate180 : چرخش ماتریس به اندازه ۱۸۰ درجه.
- rotate270 : چرخش ماتریس به اندازه ۲۷۰ درجه در جهت عقربه‌های ساعت (معادل ۹۰ درجه خلاف جهت عقربه‌های ساعت).

قانون پوشش (Masking): در حین چرخش، پس از محاسبه موقعیت جدید (newRow, newCol) برای هر عنصر، اگر مجموع (newRow + newCol) زوج باشد، مقدار آن عنصر با 1- جایگزین می‌شود (پوشش داده می‌شود). در غیر این صورت، مقدار اصلی حفظ می‌شود.

مدیریت آرایه نامنظم:

- ماتریس دارای سطرهایی با طول‌های مختلف است. هنگام چرخش، مقادیر موجود فقط به موقعیت‌های معتبر در نتیجه نگاشت می‌شوند. موقعیت‌های ناموجود حذف می‌شوند، بنابراین نتیجه نیز یک آرایه نامنظم خواهد بود.
- برای چرخش ۹۰ درجه: تعداد سطرهای نتیجه برابر با حداکثر تعداد ستون‌های ماتریس اصلی است. طول هر سطر جدید برابر با تعداد سطرهای اصلی است که در آن شماره ستون، مقداری داشته‌اند.
- برای چرخش ۱۸۰ درجه: نتیجه دارای همان تعداد سطر اصلی است. عناصر هر سطر معکوس می‌شوند و سطرها به ترتیب معکوس قرار می‌گیرند.
- برای چرخش ۲۷۰ درجه: منطق چرخش ۹۰ درجه را به صورت معکوس منعکس می‌کند.

قالب خروجی:

- ابتدا آرایه اصلی چاپ شود.
- سپس هر آرایه چرخش‌یافته با پوشش اعمال‌شده، با برچسب مشخص چاپ شود.

مثال خروجی:

Original:

[1, 2, 3]

[4, 5]

[6, 7, 8, 9]

90 degrees rotated with mask:

[-1, 4, -1]

[7, -1, 2]

[-1, 3]

[9]

180 degrees rotated with mask:

[-1, 8, -1, 6]

[5, -1]

[-1, 2, -1]

270 degrees rotated with mask:

[-1]

[3, -1]

[-1, 5, -1]

[1, -1, 6]

محدودیت‌ها:

- $1 \leq N \leq 100$
- $1 \leq \text{طول هر رشته} \leq 100$
- هر رشته فقط شامل ارقام '0' تا '9' می‌باشد.
- به جز `java.util.Scanner` از هیچ `import` دیگری از `java.util.*` استفاده نکنید.

میراث چهار رنگ

- محدودیت زمان: ۱.۵ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت
- سطح: متوسط
- طراح: آریا زریاب

در اعماق یک کتابخانه‌ی متروکه، آریا با یک لوح سنگی باستانی روبرو شده است. این لوح، شبکه‌ای به ابعاد $n \times m$ است که هر خانه‌ی آن با یکی از چهار رنگ جادویی $\{1, 2, 3, 4\}$ پر شده است.

افسانه‌ها می‌گویند این لوح، طلسمی دارد که برای شکستن آن، باید رنگ‌های آن را بازآرایی کرد. نگهبان لوح، به آریا می‌گوید که برای خنثی کردن طلسم، باید تمام خانه‌های لوح را دوباره رنگ آمیزی کنی، اما قوانین سخت‌گیرانه‌ای برای این بازآرایی وجود دارد:

۱. **قانون تقدس:** هیچ خانه‌ای نباید به رنگ اصلی و اولیه‌ی خودش باقی بماند. (اگر خانه‌ای در ابتدا رنگش c بوده، رنگ جدید باید متفاوت از c باشد).
۲. **قانون همنشینی:** هیچ دو خانه‌ی مجاور (بالا، پایین، چپ، راست) نباید رنگ یکسانی داشته باشند.
۳. **قانون محدودیت:** تو فقط به همان ۴ رنگ جادویی $\{1, 2, 3, 4\}$ دسترسی داری.

آریا نگران است که آیا راه حلی وجود دارد یا نه. او از تو می‌خواهد که برنامه‌ای بنویسی تا این طلسم را باطل کنی!

ورودی

۱. خط اول شامل دو عدد صحیح n و m است که ابعاد شبکه را نشان می‌دهند.
۲. سپس n خط می‌آید که هر خط شامل m عدد متشکل از اعداد $\{1, 2, 3, 4\}$ است که نمایانگر رنگ‌های اولیه‌ی لوح است. (بدون فاصله از هم)

$$1 \leq n, m \leq 2 \times 10^3$$

خروجی

اگر راهی برای رنگ آمیزی مجدد طبق قوانین وجود دارد، باید n خط که در هر خط m عدد (بدون فاصله) که رنگ های جدید را نشان می دهد، چاپ شود. در غیر این صورت باید کلمه Impossible چاپ شود.

مثال

ورودی نمونه ۱

```
2 3
122
311
```

خروجی نمونه ۱

```
431
142
```

جستجوی کلمه نامنظم

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت
- سطح: سخت
- طراح: مهدی افشاری

یک شبکه‌ی دو بعدی نامنظم (ragged) از حروف بزرگ انگلیسی ('A' تا 'Z') به شما داده می‌شود. در این شبکه، طول سطرها ممکن است با یکدیگر متفاوت باشد.

شما باید مشخص کنید که آیا یک **کلمه‌ی هدف** (رشته‌ای از حروف بزرگ) در این شبکه وجود دارد یا خیر.

قوانین حرکت:

- حرکت از هر خانه به خانه‌های مجاور آن در چهار جهت **بالا، پایین، چپ، راست** مجاز است.
- هر خانه فقط یک بار در مسیر جستجو قابل استفاده است (نمی‌توان از یک خانه دوبار استفاده کرد).
- حرکت به خانه‌هایی که وجود ندارند (به دلیل نامنظم بودن شبکه) **غیرمجاز** است.

ورودی

- ورودی شامل خط‌های زیر است:
 ۱. خط اول: شامل عدد صحیح n است که نشان‌دهنده‌ی تعداد سطرها در شبکه است.
 ۲. در n خط بعدی: n رشته که هر رشته نشان‌دهنده‌ی یک سطر از شبکه می‌باشد. طول هر رشته می‌تواند متغیر باشد.
 ۳. در خط آخر کلمه مورد نظر داده شده است.

محدودیت‌ها

- $1 \leq n \leq 100$
- تمام حروف: حروف بزرگ انگلیسی ('A' تا 'Z')

خروجی

یک مقدار true یا false چاپ کنید:

- true اگر کلمه‌ی هدف در شبکه وجود داشته باشد.
- false در غیر این صورت.

مثال‌ها

مثال ۱

ورودی:

```
3
ABC
DE
FGHI
ABE
```

خروجی:

```
true
```

توضیح: مسیر $A(0,0) \rightarrow B(0,1) \rightarrow E(1,0)$ معتبر است.

مثال ۲

ورودی:

```
3
ABC
DE
FGHI
CBED
```

خروجی:

true

توضیح: مسیر $C(0,2) \rightarrow B(0,1) \rightarrow E(1,1) \rightarrow D(1,0)$ معتبر است.

مثال ۳

ورودی:

3

ABC

DE

FGHI

IHE

خروجی:

false

داستان سرزمین تتریس (امتیازی)

- محدودیت زمان: ۱ ثانیه
- محدودیت حافظه: ۲۵۶ مگابایت
- سطح: سخت
- طراح: امیرحسین علی پور

مقدمه

تتریس یک بازی پازلی کلاسیک است که در آن اشکال هندسی‌ای به نام **تترومینو (Tetromino)** از بالای یک صفحهٔ مستطیلی سقوط می‌کنند.

وقتی یک قطعه روی صفحه قرار می‌گیرد، اگر یک **ردیف افقی کامل پر شود**، آن ردیف حذف می‌شود و تمام ردیف‌های بالاتر یک خانه به پایین منتقل می‌شوند.

در این مسئله باید یک **شبه‌ساز تتریس** پیاده‌سازی کنید که در آن قطعات در بالای صفحه ظاهر می‌شوند، می‌توانند به چپ یا راست حرکت کنند و سپس مستقیم به پایین سقوط کرده و در جای خود قفل شوند.

ابعاد صفحهٔ بازی

صفحهٔ بازی دارای:

• ۱۵ ردیف

• ۹ ستون

است.

```

. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .

```

```

. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .

```

تعریف خانه‌ها:

• . → خانهٔ خالی

• 1 → خانهٔ پر

در ابتدا کل صفحه خالی است.

انواع قطعات

۷ نوع تترومینو استاندارد وجود دارد.

هر قطعه به صورت یک آرایهٔ دوبعدی از 1 و . نمایش داده می‌شود.

در ادامه شکل قطعات در حالت اولیه (چرخش ۰ درجه) آمده است.

قطعه 0 (مربع)

```

1 1
1 1

```

قطعه 1 (خطی)

```

1 1 1 1

```

قطعه T


```

. 1 .
1 1 1

```

قطعه S

```

. 1 1
1 1 .

```

قطعه Z

```

1 1 .
. 1 1

```

قطعه J

```

1 . .
1 1 1

```

قطعه L

```

. . 1
1 1 1

```

حالت‌های چرخش

هر قطعه می‌تواند با زاویه‌های زیر بچرخد:

- 0°
- 90°
- 180°
- 270°

قطعه 0

در تمام چرخش‌ها یکسان است.

0°	90°	180°	270°
1 1	1 1	1 1	1 1
1 1	1 1	1 1	1 1

قطعه 1

0°	90°	180°	270°
1 1 1 1	1	1 1 1 1	1
	1		1
	1		1
	1		1

قطعه T

0°	90°	180°	270°
. 1 .	1 .	1 1 1	. 1
1 1 1	1 1	. 1 .	1 1
	1 .		. 1

قطعه S

0°	90°	180°	270°
. 1 1	1 .	1 1 .	. 1
1 1 .	1 1	. 1 1	1 1
	. 1		1 .

قطعه Z

0°	90°	180°	270°
1 1 .	. 1	. 1 1	1 .
. 1 1	1 1	1 1 .	1 1
	1 .		. 1

قطعه J

0°	90°	180°	270°
1 . .	1 1	1 1 1	. 1
1 1 1	1 .	. . 1	. 1
	1 .		1 1

قطعه L

0°	90°	180°	270°
. . 1	1 .	1 1 1	1 1
1 1 1	1 .	1 . .	. 1
	1 1		. 1

قوانین حرکت افقی

وقتی یک قطعه ظاهر می‌شود، قبل از سقوط می‌تواند به چپ یا راست حرکت کند.

حرکت در ردیف صفر (بالای صفحه) انجام می‌شود.

محدودیت حرکت

قطعه نباید از مرز صفحه خارج شود.

- بیشترین حرکت به چپ: تا زمانی که اولین ستون قطعه به ستون 0 برسد
- بیشترین حرکت به راست: تا زمانی که آخرین ستون قطعه به ستون 8 برسد

اگر قطعه‌ای با عرض W در ستون C ظاهر شود:

```
max_left_shift = C
max_right_shift = 8 - (C + W - 1)
```

اگر تعداد حرکت‌های درخواستی بیشتر از حد مجاز باشد، تا حد مجاز محدود می‌شود.

صورت مسئله

دنباله‌ای از قطعات داده می‌شود.

برای هر قطعه:

- نوع قطعه
- زاویهٔ چرخش
- تعداد حرکت به چپ
- تعداد حرکت به راست

باید سقوط قطعات را روی صفحهٔ 9×15 شبیه‌سازی کنید و در نهایت وضعیت نهایی صفحه را چاپ کنید.

فرمت ورودی

```
N
type rotation leftShift rightShift
type rotation leftShift rightShift
...
```

توضیح

- $N \rightarrow$ تعداد قطعات (۱ تا ۱۰۰)

برای هر قطعه:

- نوع قطعه:

O I T S Z J L

- چرخش:

270 ، 180 ، 90 ، 0

- تعداد حرکت به چپ
- تعداد حرکت به راست

فرمت خروجی

در نهایت صفحه ۹×۱۵ چاپ شود.

هر ردیف در یک خط:

```
. . . . .
. . . . .
...
```

قوانین شبیه سازی

انتخاب شکل قطعه

با توجه به نوع قطعه و زاویه چرخش، شکل درست قطعه انتخاب می شود.

ظاهر شدن و حرکت

قطعه در ابتدا در بالای صفحه ظاهر می شود.

```
spawn column = (9 - piece_width) / 2
```

سپس:

- حرکت های چپ اعمال می شوند
- حرکت های راست اعمال می شوند
- حرکت های خارج از محدوده نادیده گرفته می شوند

سقوط قطعه

قطعه به صورت عمودی سقوط می‌کند.

تا زمانی که:

- با قطعهٔ دیگر برخورد نکند
- از صفحه خارج نشود

قفل شدن

وقتی قطعه دیگر نتواند پایین‌تر برود:

خانه‌های 1 آن روی صفحه ثبت می‌شوند.

اگر قطعه از ابتدا روی بلوک موجود قرار بگیرد → بازی تمام می‌شود.

حذف ردیف کامل

پس از قرار گرفتن قطعه:

از پایین صفحه بررسی می‌شود.

اگر ردیفی کاملاً پر باشد:

- آن ردیف حذف می‌شود
- همهٔ ردیف‌های بالا یک خانه پایین می‌آیند
- ردیف صفر خالی می‌شود

این کار تا زمانی که ردیف پر وجود داشته باشد ادامه پیدا می‌کند.

مثال‌ها

ورودی 1

```

3
0 0 0 0
I 0 0 3
T 90 2 0

```

خروجی 1

```

. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. 1 . . . . .
. 1 1 1 1 . . . .
. 1 . 1 1 1 1 1 1

```

ورودی 2

```

2
0 0 10 0
I 0 0 20

```

خروجی 2

```

. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . . . .

```

```
. . . . . . . . .  
. . . . . . . . .  
. . . . . . . . .  
. . . . . . . . .  
. . . . . . . . .  
. . . . . . . . .  
1 1 . . . . . . .  
1 1 . . . 1 1 1 1
```